TECHNICAL DEEP-DIVE

Control Flow · Functions · Libraries · APIs

LLM-Based Threat Modeling System

Task 1 (RAG Pipeline) · Task 2 (KeyBERT + NVD + RAG)

Every function documented · Every library explained · Every API call detailed

Table of Contents

A Libraries & Installation

Every library used, why it was chosen, and what it provides

A1. LangChain

LangChain is an open-source framework for building applications powered by large language models. It provides standardised interfaces for loading documents, splitting text, creating embeddings, querying vector stores, and chaining LLM calls. In this system it serves as the orchestration backbone for both Task 1 and Task 2.

Install command	Purpose
pip install langchain langchain-community	Core LangChain + community integrations

LangChain modules used — function-by-function

PyPDFDirectoryLoader [langchain_community.document_loaders]

Signature	`PyPDFDirectoryLoader(path: str)`
What it does	Scans a directory, opens every .pdf file, extracts text page-by-page using pypdf, and returns a list of Document objects. Each Document carries the page text plus metadata: {'source': 'filename.pdf', 'page': 3}.
Returns	List[Document] — each Document has .page_content (str) and .metadata (dict).
Used in	load_and_chunk_documents() — called once at startup to ingest all design documents.

Parameters

Parameter	Description
path	String path to the directory containing PDF files. All .pdf files found recursively are loaded.

Code example

```
loader = PyPDFDirectoryLoader('./design_documents/')

docs = loader.load()   # returns e.g. 847 page-Document objects
```

RecursiveCharacterTextSplitter [langchain.text_splitter]

Signature	`RecursiveCharacterTextSplitter(chunk_size, chunk_overlap, separators)`
What it does	Splits long text into smaller overlapping chunks. Tries to split on paragraph breaks (\n\n), then newlines (\n), then spaces, then individual characters — whichever fits within chunk_size. The overlap ensures a sentence split across chunk boundaries is fully present in both chunks, preserving context.
Returns	List[Document] — same structure as input but with text split into chunks.
Used in	load_and_chunk_documents() — called after PyPDFDirectoryLoader.

Parameters

Parameter	Description
chunk_size	Maximum characters per chunk. Set to 500 in this system (paper specification).
chunk_overlap	Number of characters shared between consecutive chunks. Set to 20. Prevents losing context at boundaries.
separators	Default: ['\n\n','\n',' ',''']. Tries each in order until chunk fits.

Code example

```
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=20)
```

```
chunks = splitter.split_documents(docs)  # e.g. 4,200 chunks from 12 docs
```

HuggingFaceEmbeddings [langchain_community.embeddings]

Signature	`HuggingFaceEmbeddings(model_name: str)`
What it does	Wraps a HuggingFace sentence-transformer model and exposes it as a LangChain-compatible embedder. When called with a list of texts, it tokenises them, runs them through the transformer, and returns dense float vectors. The model all-MiniLM-L6-v2 produces 384-dimensional vectors capturing semantic meaning.
Returns	Embeddings object with .embed_documents(texts) and .embed_query(text) methods.
Used in	create_embeddings() — model loaded once and reused for both indexing and query-time embedding.

Parameters

Parameter	Description
model_name	HuggingFace model identifier. 'sentence-transformers/all-MiniLM-L6-v2' — fast BERT model, 384-dim, good for semantic search.

Code example

```
embeddings = HuggingFaceEmbeddings(model_name='sentence-transformers/all-MiniLM-L6-v2')
```

```
# Internally: tokenise → BERT forward pass → mean pool → 384-dim vector
```

Pinecone.from_documents() [langchain_community.vectorstores]

Signature	`Pinecone.from_documents(documents, embedding, index_name)`
What it does	Takes a list of LangChain Documents and an embedding model, computes the embedding for each document's page_content, then upserts all (vector, metadata) pairs into the named Pinecone index in batches. This is the indexing step — after this call, the index is ready to answer semantic similarity queries.
Returns	Pinecone VectorStore object with .as_retriever() and .similarity_search() methods.
Used in	build_vector_database() in both Task 1 and Task 2.

Parameters

Parameter	Description
documents	List[Document] — the chunked documents to index.
embedding	HuggingFaceEmbeddings instance — used to compute each chunk's vector.
index_name	String — the Pinecone index to write into. Must already exist.

Code example

```
vector_store = LangchainPinecone.from_documents(

  chunks, embeddings, index_name='threat-modeling-task1')
```

RetrievalQA.from_chain_type() [langchain.chains]

Signature	`RetrievalQA.from_chain_type(llm, chain_type, retriever, return_source_documents)`
What it does	Builds a question-answering chain that (1) retrieves relevant chunks from a vector store using the retriever, (2) concatenates them into a context

	string, (3) wraps them in a prompt template, and (4) passes the augmented prompt to the LLM. The 'stuff' chain type simply concatenates all retrieved chunks into a single context — suitable when top-k chunks fit in the LLM context window.
Returns	RetrievalQA chain object. Call with {'query': question} → returns {'result': answer, 'source_documents': [...]}.
Used in	build_rag_chain() — creates the full RAG pipeline in both tasks.

Parameters

Parameter	Description
llm	HuggingFacePipeline — the Llama 2 model that generates the final answer.
chain_type	'stuff' — concatenate all retrieved chunks into context. Alternatives: 'map_reduce', 'refine'.
retriever	VectorStore.as_retriever(search_kwargs={'k':5}) — retrieves top-5 semantically similar chunks.
return_source_documents	True — include the retrieved Document objects in the output dict for citation.

Code example

```
rag = RetrievalQA.from_chain_type(
  llm=llm, chain_type='stuff',
  retriever=vs.as_retriever(search_kwargs={'k':5}),
  return_source_documents=True)
result = rag({'query': 'How is data encrypted?'})
```

HuggingFacePipeline [langchain_community.llms]

Signature	`HuggingFacePipeline(pipeline: transformers.Pipeline)`
What it does	Wraps a HuggingFace transformers pipeline so it can be used as a LangChain LLM. Converts string prompts to model input, runs inference, and returns generated text. Necessary because LangChain chains expect an object with a standard __call__(prompt) interface.
Returns	LangChain-compatible LLM object.
Used in	load_llm() — wraps the Llama 2 HuggingFace pipeline.

Parameters

Parameter	Description
pipeline	A transformers.pipeline object with task='text-generation' pointing to Llama 2.

Code example

```
pipe = pipeline('text-generation', model=model, tokenizer=tok, max_new_tokens=512)
llm  = HuggingFacePipeline(pipeline=pipe)
```

A2. Pinecone

Pinecone is a managed vector database service. Unlike relational databases that query by exact value, Pinecone finds vectors by semantic similarity (cosine distance, dot product, or Euclidean). It is used in both Task 1 (design document chunks) and Task 2 (CVE descriptions) as the knowledge base that enables RAG retrieval.

Install	API key	Docs
pip install pinecone-client	PINECONE_API_KEY env var	docs.pinecone.io

Pinecone functions used

pinecone.init()

Signature	`pinecone.init(api_key: str, environment: str)`
What it does	Authenticates with the Pinecone service and sets the global connection context. Must be called before any index operation.

Parameter	Description
api_key	Your Pinecone API key (from console.pinecone.io). Never hardcode — use environment variables.
environment	Pinecone deployment region, e.g. 'us-east-1-aws'. Found in your Pinecone console.

```
pinecone.init(api_key=os.getenv('PINECONE_API_KEY'), environment='us-east-1-aws')
```

pinecone.list_indexes()

Signature	`pinecone.list_indexes() → List[str]`
What it does	Returns a list of index names currently in your Pinecone project. Used to check if an index already exists before attempting to create it (avoids error on duplicate creation).

```
if 'threat-modeling-task1' not in pinecone.list_indexes():

    pinecone.create_index(...)
```

pinecone.create_index()

Signature	`pinecone.create_index(name, dimension, metric)`
What it does	Creates a new Pinecone index. The dimension must exactly match the output dimension of your embedding model (384 for all-MiniLM-L6-v2). The metric defines how similarity is computed during queries.

Parameter	Description
name	String index name. Must be lowercase, alphanumeric, hyphens allowed.
dimension	Integer — must match embedding model output. 384 for all-MiniLM-L6-v2.
metric	Similarity metric. 'cosine' used here — measures angle between vectors, ignoring magnitude.

```
pinecone.create_index('threat-modeling-task1', dimension=384, metric='cosine')
```

> **Why cosine similarity?**
>
> Cosine similarity measures the angle between two vectors regardless of their magnitude. Two semantically similar chunks will point in nearly the same direction in 384D space even if one is much longer than the other. This makes it ideal for text — a short sentence and a paragraph about the same topic will have nearly the same cosine similarity score.

A3. HuggingFace Transformers

The transformers library provides pre-trained neural network models (BERT, Llama 2, etc.) and the tokenizers, training utilities, and inference pipelines to use them. In this system it is used to load Llama 2 Chat (the generative LLM) and expose it as a text-generation pipeline.

Install	GPU requirement
pip install transformers accelerate torch	T4 GPU (Google Colab) or equivalent — Llama 2 7B needs ~14GB VRAM at float16

HuggingFace functions used

AutoTokenizer.from_pretrained()

Signature	`AutoTokenizer.from_pretrained(model_name, token)`
What it does	Downloads and caches the tokenizer for the specified model. For Llama 2, this is a SentencePiece BPE tokenizer with a vocabulary of 32,000 tokens. The tokenizer converts raw strings to token ID tensors and back, handling special tokens (<s>, </s>, [INST], etc.) required by Llama 2 Chat's instruction format.

Parameter	Description
model_name	HuggingFace model path: 'meta-llama/Llama-2-7b-chat-hf' — the 7B instruction-tuned chat model.
token	HuggingFace access token — required because Llama 2 is a gated model (must accept Meta's licence).

```
tokenizer = AutoTokenizer.from_pretrained(
  'meta-llama/Llama-2-7b-chat-hf', token=HF_TOKEN)
```

AutoModelForCausalLM.from_pretrained()

Signature	`AutoModelForCausalLM.from_pretrained(model_name, token, torch_dtype, device_map)`
What it does	Downloads the full Llama 2 model weights (~13.5GB at float16) and loads them onto the GPU. AutoModelForCausalLM is a causal language model — it predicts the next token given all previous tokens. Llama 2 Chat is fine-tuned on instruction-following using RLHF, making it suitable for Q&A tasks.

Parameter	Description

model_name	'meta-llama/Llama-2-7b-chat-hf' — 7B parameter variant. Paper also uses 13B variant.
torch_dtype	torch.float16 — half-precision to halve memory use (acceptable quality loss for inference).
device_map	'auto' — automatically distributes model layers across available GPUs/CPU.

```
model = AutoModelForCausalLM.from_pretrained(
  'meta-llama/Llama-2-7b-chat-hf', token=HF_TOKEN,
  torch_dtype=torch.float16, device_map='auto')
```

pipeline() [transformers]

Signature	`pipeline(task, model, tokenizer, max_new_tokens, temperature, do_sample)`
What it does	Creates a high-level inference pipeline. For 'text-generation', it handles the full loop: tokenise input → run model.generate() → decode output tokens back to string. It abstracts sampling strategies, attention masks, padding, and decoding.

Parameter	Description
task	'text-generation' — tells pipeline to generate text continuation from a prompt.
model	The loaded AutoModelForCausalLM instance.
tokenizer	The matching AutoTokenizer instance.
max_new_tokens	512 — maximum tokens to generate per response. Controls response length.
temperature	0.1 — low temperature = more deterministic, higher = more creative. 0.1 chosen for factual Q&A.
do_sample	True — enables sampling (otherwise greedy decoding always picks highest probability token).

```
pipe = pipeline(
  'text-generation', model=model, tokenizer=tokenizer,
  max_new_tokens=512, temperature=0.1, do_sample=True)
```

A4. KeyBERT

KeyBERT is an NLP keyword extraction library that uses BERT embeddings and cosine similarity to find the keywords and keyphrases in a document most semantically representative of the document as a whole. Unlike TF-IDF (which counts frequencies), KeyBERT picks terms that are close to the document's meaning in vector space.

Install
pip install keybert sentence-transformers

KeyBERT functions used

KeyBERT() — constructor

Signature	`KeyBERT(model='all-MiniLM-L6-v2')`
What it does	Initialises KeyBERT with a sentence-transformer model. The model encodes both the full document and individual candidate keyphrases into vectors. Keyphrases whose vectors are most similar (highest cosine similarity) to the WHOLE document vector are selected as the most representative keywords.
Default model	all-MiniLM-L6-v2 — same BERT model used for Pinecone embeddings, ensuring consistency.

kw_model.extract_keywords()

Signature	`extract_keywords(docs, keyphrase_ngram_range, stop_words, top_n, use_mmr, diversity)`
What it does	Extracts the most representative keywords from the input text. It generates candidate n-grams, embeds both the document and each candidate, then ranks candidates by cosine similarity to the document embedding. MMR (Maximal Marginal Relevance) diversifies the selection to avoid redundant keyphrases.
Returns	List of (keyword, score) tuples, e.g. [('load value injection', 0.74), ('TDX attestation', 0.68), ...]

Parameter	Value used	Explanation
keyphrase_ngram_range	(1, 3)	Extract 1-word, 2-word, AND 3-word phrases. 'load value injection' is far more specific than just 'injection'.
stop_words	NLTK + expert list	Remove common English words AND generic security terms before extraction. See expert stopwords section.
top_n	15	Return top 15 most representative keyphrases per document.

use_mmr	True	Apply Maximal Marginal Relevance to diversify results — avoids 5 variants of 'encryption'.
diversity	0.5	MMR balance: 0 = pure relevance, 1 = pure diversity. 0.5 balances both.

```
kw_model = KeyBERT()
keywords = kw_model.extract_keywords(
  full_text,
  keyphrase_ngram_range=(1, 3),
  stop_words=combined_stopwords,   # NLTK + expert list
  top_n=15,
  use_mmr=True,
  diversity=0.5
)
# Returns: [('load value injection', 0.74), ('trust domain extensions', 0.71), ...]
```

A5. NLTK — Natural Language Toolkit

NLTK is Python's foundational NLP library. In this system it provides two specific tools: (1) a standard English stopword list used to filter out common words before KeyBERT extraction, and (2) the PorterStemmer for reducing words to root forms before NVD API querying to avoid duplicate searches.

Install	Data download
pip install nltk	nltk.download('stopwords') | nltk.download('punkt')

NLTK functions used

stopwords.words('english')

Signature	`from nltk.corpus import stopwords stopwords.words('english')`
What it does	Returns a list of 179 common English words that carry no semantic meaning for keyword extraction: 'the', 'and', 'in', 'it', 'are', 'that', 'start', 'but', 'to', 'with'... Removing these before KeyBERT extraction ensures the model focuses on domain-specific content words.
Why remove them?	If stopwords are not removed, KeyBERT may rank 'the' or 'and' as high-frequency candidates even though they are meaningless for NVD queries.

PorterStemmer().stem()

Signature	`stemmer = PorterStemmer()  stemmer.stem(word: str) → str`
What it does	Reduces a word to its root form by removing suffixes using rule-based heuristics. 'authentication' → 'authent', 'authenticating' → 'authent', 'authenticated' → 'authent'. Ensures that all inflected forms of the same word generate only ONE NVD API query, avoiding duplicate CVE results.
Used in	extract_keywords_from_pdf() — after KeyBERT extraction, each keyword's first word is stemmed and checked against a seen_stems set to deduplicate.

```
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer


nltk_sw  = set(stopwords.words('english'))       # 179 words: the, and, in ...
all_sw   = list(nltk_sw | EXPERT_STOPWORDS)      # merge with expert list


stemmer  = PorterStemmer()
seen_stems, keywords = set(), []
for kw, score in raw_keywords:
    stem = stemmer.stem(kw.split()[0])            # stem first word of phrase
    if stem not in seen_stems:
        seen_stems.add(stem)                      # deduplicate
        keywords.append(kw)
```

A6. pypdf (PDF text extraction)

pypdf is a pure-Python library for reading PDF files without any external dependencies. It is used in Task 2 to extract the raw text from design documents before feeding them to KeyBERT for keyword extraction.

Install
pip install pypdf

PdfReader

Signature	`from pypdf import PdfReader reader = PdfReader(path) reader.pages[n].extract_text()`
What it does	Opens a PDF file, parses its binary structure, and provides access to each page as a PageObject. The extract_text() method attempts to reconstruct the text content from the PDF's character stream. Quality varies with PDF type — text-based PDFs extract cleanly; scanned images require OCR (not used here).
Used in	extract_keywords_from_pdf() — reads each design document PDF and joins all page texts into a single string for KeyBERT.

```
from pypdf import PdfReader


reader    = PdfReader('./design_documents/intel_tdx.pdf')
full_text = ' '.join(page.extract_text() or '' for page in reader.pages)

# full_text is now a single string of all PDF content

# 'or ""' handles pages with no extractable text (images, blank pages)
```

A7. Requests (NVD API HTTP client)

The requests library is Python's standard HTTP client. It is used in Task 2 to programmatically query the NIST National Vulnerability Database (NVD) REST API v2.0 for CVE records matching the keywords extracted by KeyBERT.

Install	NVD API key	NVD API base URL
pip install requests	optional but recommended (higher rate limit: 50 req/30s vs 5 req/30s)	https://services.nvd.nist.gov/rest/json/cves/2.0

requests.get() — NVD API call

Signature	`requests.get(url, params, headers, timeout) → Response`
What it does	Sends an HTTP GET request to the NVD API with query parameters. The API returns a JSON response containing CVE records matching the keyword. Each call is rate-limited — a 0.6-second sleep is added between calls to stay within NVD's limits.
params dict	{'keywordSearch': keyword, 'resultsPerPage': 100} — tells NVD to search all CVE descriptions for the keyword and return up to 100 results.
headers dict	{'apiKey': api_key} — if an NVD API key is provided, attach it here for higher rate limits.

timeout	30 seconds — prevents the script from hanging indefinitely if NVD is slow.
Error handling	Wrapped in try/except requests.exceptions.RequestException — network errors, timeouts, and 5xx responses are caught and logged without crashing the full keyword loop.

```
import requests, time


def query_nvd_api(keyword, api_key=''):
    url     = 'https://services.nvd.nist.gov/rest/json/cves/2.0'
    headers = {'apiKey': api_key} if api_key else {}
    params  = {'keywordSearch': keyword, 'resultsPerPage': 100}
    try:
        resp = requests.get(url, params=params, headers=headers, timeout=30)
        resp.raise_for_status()          # raises HTTPError for 4xx/5xx
        data = resp.json()
        return data.get('vulnerabilities', [])
    except requests.exceptions.RequestException as e:
        print(f'NVD error for {keyword}: {e}')
        return []


time.sleep(0.6)   # rate limit between calls
```

B Task 1 — Document Understanding (MQ1)

Complete control flow, every function, every line explained

B1. Complete Control Flow — Task 1

Task 1 answers MQ1: 'What are we working on?' It builds a RAG pipeline over design documents so security engineers can ask natural-language questions and receive grounded, source-cited answers about the product architecture.

```
main()  [__main__]
  │
  ├── load_and_chunk_documents(DOCS_DIR)
  │         │
  │         ├── PyPDFDirectoryLoader(docs_dir).load()
  │         │     └── Returns: List[Document]  (e.g. 847 page-documents)
  │         │
  │         └── RecursiveCharacterTextSplitter(
  │               chunk_size=500, chunk_overlap=20
  │             ).split_documents(documents)
  │               └── Returns: List[Document]  (e.g. 4,200 chunks)
  │
  ├── create_embeddings()
  │         └── HuggingFaceEmbeddings('sentence-transformers/all-MiniLM-L6-v2')
  │               └── Returns: Embeddings object (384-dim BERT model)
  │
  ├── build_vector_database(chunks, embeddings)
  │         ├── pinecone.init(PINECONE_API_KEY, PINECONE_ENV)
  │         ├── if INDEX_NAME not in pinecone.list_indexes():
  │         │       pinecone.create_index(INDEX_NAME, dim=384, metric='cosine')
  │         └── LangchainPinecone.from_documents(chunks, embeddings, index_name)
  │               └── Returns: VectorStore object
  │
  ├── load_llm()
  │         ├── AutoTokenizer.from_pretrained(LLM_MODEL, token=HF_TOKEN)
  │         ├── AutoModelForCausalLM.from_pretrained(
  │         │     LLM_MODEL, torch_dtype=float16, device_map='auto')
  │         ├── pipeline('text-generation', model, tokenizer,
  │         │     max_new_tokens=512, temperature=0.1)
  │         └── HuggingFacePipeline(pipe)
  │               └── Returns: LangChain LLM object
  │
  ├── build_rag_chain(vector_store, llm)
  │         ├── vector_store.as_retriever(search_kwargs={'k': 5})
  │         └── RetrievalQA.from_chain_type(
  │               llm, chain_type='stuff', retriever,
  │               return_source_documents=True)
  │               └── Returns: RAG chain object
  │
  └── for question in TASK1_QUESTIONS:          # 6 questions per document
            │
            ├── query_base_llm(llm, question)    # comparison baseline
            │     └── llm('You are a security engineer. ' + question)
            │           └── Returns: str (raw Llama 2 output ~253 words avg)
            │
            └── query_rag_llm(rag_chain, question)
```

```
                    └── rag_chain({'query': question})
                          ├── embed question → 384-dim vector
                          ├── Pinecone ANN search → top-5 chunks
                          ├── Build prompt: context + question
                          ├── Llama 2 generates grounded answer (~74 words avg)
                          └── Returns: {answer, sources: [{source, page}]}
```

Key insight:

The control flow splits at 'query_base_llm vs query_rag_llm' — this is the core experiment of the paper. Both use the same Llama 2 model. The ONLY difference is whether retrieved document chunks are injected into the prompt. This isolates the contribution of RAG.

B2. load_and_chunk_documents()

Signature	`load_and_chunk_documents(docs_dir: str) → List[Document]`
Purpose	Loads all PDF design documents from a directory, extracts their text, and splits them into overlapping fixed-size chunks ready for embedding. This is Step 1-1 and Step 1-2 of the Task 1 pipeline.
Returns	List[Document] — each Document has .page_content (one 500-char chunk) and .metadata (source file + page).
Why it exists	Security engineers cannot read 60-page PDFs manually for every question. This function replaces that manual process — all documents are loaded and chunked in seconds.

Internal control flow

```
1. PyPDFDirectoryLoader(docs_dir)
     ↓  scans directory for *.pdf files
     ↓  opens each PDF with pypdf
     ↓  extracts text per page
     ↓  attaches metadata: {source, page}
2. loader.load() → List[Document]
3. RecursiveCharacterTextSplitter(
     chunk_size=500, chunk_overlap=20
   )
4. splitter.split_documents(documents) → List[Document] (chunks)
Returns chunks
```

Parameters

Parameter	Description
docs_dir	Path to folder containing PDF design documents. Any .pdf in this folder is loaded.

Code

```
def load_and_chunk_documents(docs_dir: str):
    loader   = PyPDFDirectoryLoader(docs_dir)
    documents = loader.load()                  # all pages from all PDFs
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=500,     # ~3-4 sentences per chunk
        chunk_overlap=20,   # 20-char overlap prevents context loss
    )
    chunks = splitter.split_documents(documents)
    return chunks
```

B3. create_embeddings()

Signature	`create_embeddings() → HuggingFaceEmbeddings`
Purpose	Loads the BERT-based sentence transformer model that converts text chunks and queries into 384-dimensional semantic vectors. The same model instance is used for both indexing (chunks) and query-time (questions), ensuring vectors are in the same space.
Returns	HuggingFaceEmbeddings — callable embedder producing 384-dim float vectors.
Why it exists	Semantic search requires all text to be in the same vector space. Using the same model for both chunks and queries ensures apples-to-apples cosine similarity comparison.

Internal control flow

```
  1. Load 'sentence-transformers/all-MiniLM-L6-v2' from HuggingFace Hub
       ↓  downloads model weights (~22MB) on first run, cached after
       ↓  wraps in LangChain HuggingFaceEmbeddings interface
  2. Returns embeddings object
  Usage: embeddings.embed_documents(texts) → List[List[float]]
         embeddings.embed_query(text)  → List[float]
```

Code

```
EMBED_MODEL = 'sentence-transformers/all-MiniLM-L6-v2'
```

```
def create_embeddings():
    embeddings = HuggingFaceEmbeddings(model_name=EMBED_MODEL)
```

```
    return embeddings

    # each .embed() call: tokenise → BERT forward pass → mean pool → 384-dim
```

B4. build_vector_database()

Signature	`build_vector_database(chunks, embeddings) → VectorStore`
Purpose	Creates (or reuses) a Pinecone index and populates it with embeddings of all document chunks. This is the 'indexing' phase of RAG — Steps 1-3 and 1-4. After this function returns, the knowledge base is ready for semantic queries.
Returns	LangchainPinecone VectorStore — provides .as_retriever() and .similarity_search() for RAG.
Why it exists	Pinecone's ANN (Approximate Nearest Neighbour) index enables sub-second similarity searches across thousands of vectors — something impossible with brute-force vector comparison.

Internal control flow

```
1. pinecone.init(api_key, environment)       # authenticate
2. Check if INDEX_NAME in pinecone.list_indexes()
     if NOT: pinecone.create_index(
                INDEX_NAME, dimension=384, metric='cosine')
3. LangchainPinecone.from_documents(
       chunks, embeddings, index_name=INDEX_NAME)
     ↓  for each chunk:
     ↓    embed chunk → 384-dim vector
     ↓    upsert (vector, metadata) into Pinecone
     ↓  metadata per chunk: {source, page, text}
4. Returns vector_store object
```

Parameters

Parameter	Description
chunks	List[Document] — output of load_and_chunk_documents().
embeddings	HuggingFaceEmbeddings — output of create_embeddings().

Code

```
def build_vector_database(chunks, embeddings):

    pinecone.init(api_key=PINECONE_API_KEY, environment=PINECONE_ENV)

    if INDEX_NAME not in pinecone.list_indexes():

        pinecone.create_index(
```

```
            INDEX_NAME, dimension=384, metric='cosine')
    vector_store = LangchainPinecone.from_documents(
        chunks, embeddings, index_name=INDEX_NAME)
    return vector_store
```

B5. load_llm()

Signature	`load_llm() → HuggingFacePipeline`
Purpose	Downloads and initialises Llama 2 Chat (7B parameters) on GPU, wraps it in a HuggingFace text-generation pipeline, and further wraps that in a LangChain-compatible interface. This is the 'brain' of the system used by both base LLM and RAG-enhanced paths.
Returns	HuggingFacePipeline — LangChain-compatible LLM. Call with llm(prompt_string) → generated text string.
Why it exists	Llama 2 Chat was chosen because it outperforms other open-source models, is on-par with closed-source models in human evals, and explicitly prioritises safety via RLHF and red-teaming.

Internal control flow

```
1. AutoTokenizer.from_pretrained(LLM_MODEL, token=HF_TOKEN)
     ↓  loads SentencePiece BPE tokenizer (vocab_size=32,000)
2. AutoModelForCausalLM.from_pretrained(
     LLM_MODEL, torch_dtype=float16, device_map='auto')
     ↓  downloads ~13.5GB weights
     ↓  loads onto T4 GPU in float16
3. transformers.pipeline(
     'text-generation', model, tokenizer,
     max_new_tokens=512, temperature=0.1, do_sample=True)
4. HuggingFacePipeline(pipe)   # LangChain wrapper
Returns: llm object
```

Code

```
LLM_MODEL = 'meta-llama/Llama-2-7b-chat-hf'
```

```
def load_llm():
    tokenizer = AutoTokenizer.from_pretrained(LLM_MODEL, token=HF_TOKEN)
    model = AutoModelForCausalLM.from_pretrained(
        LLM_MODEL, token=HF_TOKEN,
        torch_dtype=torch.float16,   # ~13.5GB on GPU
```

```
        device_map='auto',           # auto-place on T4 GPU
    )
    pipe = pipeline('text-generation', model=model, tokenizer=tokenizer,
        max_new_tokens=512, temperature=0.1, do_sample=True)
    return HuggingFacePipeline(pipeline=pipe)
```

B6. query_base_llm()

Signature	`query_base_llm(llm, question: str) → str`
Purpose	Queries Llama 2 WITHOUT any retrieved context. Used as the baseline for comparison against the RAG-enhanced path. The LLM must rely entirely on what it memorised during pre-training — no product-specific information is available.
Returns	str — raw Llama 2 response. Typically verbose (~253 words), generic, may hallucinate product-specific details.
Why it exists	The paper compares base LLM vs RAG-enhanced LLM on every query to isolate RAG's contribution. query_base_llm provides the control group.

Internal control flow

```
1. Build prompt: role prompt + question
2. llm(prompt) → Llama 2 forward pass → token sampling → str
Returns: generated string (~253 words average)
```

Parameters

Parameter	Description
llm	HuggingFacePipeline — the loaded Llama 2 model.
question	Natural language threat modeling question, e.g. 'What components are involved in the design?'

Code

```
def query_base_llm(llm, question: str) -> str:
    prompt = (
        'You are a security engineer performing threat modeling. '
        f'Answer the following question:\n\n{question}'
    )
```

```
    return llm(prompt)    # no context injected — relies on training data
```

B7. build_rag_chain()

Signature	`build_rag_chain(vector_store, llm) → RetrievalQA`
Purpose	Assembles the full RAG pipeline by connecting the Pinecone vector store (retriever) to Llama 2 (generator) through LangChain's RetrievalQA chain. When queried, this chain automatically retrieves relevant document chunks and injects them into the prompt.
Returns	RetrievalQA chain. Call with {'query': question} → {'result': answer, 'source_documents': [Document, ...]}
Why it exists	'stuff' chain type concatenates all retrieved chunks before the question. This works because 5 chunks × 500 chars = 2,500 chars — well within Llama 2's 4,096 token context window.

Internal control flow

```
1. vector_store.as_retriever(search_kwargs={'k': 5})
     ↓  creates a Retriever that finds top-5 most similar chunks
     ↓  similarity metric: cosine distance in Pinecone
2. RetrievalQA.from_chain_type(
     llm=llm,
     chain_type='stuff',              # concatenate all 5 chunks
     retriever=retriever,
     return_source_documents=True     # include citations
   )
Returns: rag_chain object
```

Parameters

Parameter	Description
vector_store	Pinecone VectorStore — the indexed document knowledge base.
llm	HuggingFacePipeline — Llama 2 Chat model.

Code

```
TOP_K = 5   # retrieve 5 most relevant chunks
```

```
def build_rag_chain(vector_store, llm):
```

```
    retriever = vector_store.as_retriever(
        search_kwargs={'k': TOP_K}
```

```
    )
    rag_chain = RetrievalQA.from_chain_type(
        llm=llm,
        chain_type='stuff',
        retriever=retriever,
        return_source_documents=True,
    )
    return rag_chain
```

B8. query_rag_llm()

Signature	`query_rag_llm(rag_chain, question: str) → dict`
Purpose	Executes the full RAG pipeline for a single question. Internally it embeds the question, retrieves the top-5 most similar chunks from Pinecone, builds an augmented prompt, sends it to Llama 2, and returns the grounded answer alongside source citations.
Returns	dict with 'answer' (str, ~74 words average) and 'sources' (list of {source, page} dicts for citation).
Why it exists	RAG-enhanced responses are more concise (74 vs 253 words), product-specific, and cite sources. When context is insufficient, the model explicitly says 'I don't know' rather than hallucinating.

Internal control flow

```
1. rag_chain({'query': question})
     ↓  embed question → 384-dim vector (same BERT model as indexing)
     ↓  Pinecone cosine search → top-5 chunk Documents
     ↓  build prompt:
          'Use ONLY the context below to answer.',
          'Context: [chunk1]\n[chunk2]\n...[chunk5]',
          'Question: {question}'
     ↓  Llama 2 generates answer from context only
     ↓  if context insufficient → 'I don't know...' (honesty)
2. Extract answer string from result['result']
3. Extract source metadata from result['source_documents']
Returns: {answer, sources: [{source, page}, ...]}
```

Parameters

Parameter	Description
rag_chain	RetrievalQA — assembled by build_rag_chain().

question	Natural language question about the system design.

Code

```
def query_rag_llm(rag_chain, question: str) -> dict:
    result = rag_chain({'query': question})
    return {
        'answer': result['result'],
        'sources': [
            {
                'source': doc.metadata.get('source', 'unknown'),
                'page':   doc.metadata.get('page', '?'),
            }
            for doc in result.get('source_documents', [])
        ],
    }
```

C

Task 2 — Vulnerability Identification (MQ2)

KeyBERT + NVD API + RAG: every function and API call documented

C1. Complete Control Flow — Task 2

Task 2 answers MQ2: 'What can go wrong?' It builds a custom, always-current CVE knowledge base from the NVD by using design-document keywords as search terms, then uses RAG to answer security-focused questions with live CVE references.

```
main()  [__main__]
  │
  ├── extract_keywords_from_all_docs(DOCS_DIR, KEYWORDS_CSV)
  │         │
  │         └── for each PDF in DOCS_DIR:
  │                 extract_keywords_from_pdf(pdf_path)
  │                     ├── PdfReader(pdf_path).pages → full_text
  │                     ├── KeyBERT().extract_keywords(
  │                     │     full_text,
  │                     │     keyphrase_ngram_range=(1,3),
  │                     │     stop_words=NLTK+expert,
  │                     │     top_n=15, use_mmr=True)
  │                     └── PorterStemmer deduplication
  │                           └── Returns: List[str]  (15 unique keyphrases)
  │         aggregate all → unique_keywords
  │         save → keywords.csv
  │         Returns: List[str]  (all unique keywords across all docs)
  │
  ├── build_cve_dataset(keywords, CVE_CSV, NVD_API_KEY)
  │         │
  │         └── for each keyword:
  │                 query_nvd_api(keyword, api_key)
  │                     ├── requests.get(NVD_URL, params={keywordSearch: kw})
  │                     ├── parse JSON response
  │                     └── extract per CVE:
  │                           cve_id, description, severity (CVSS),
  │                           exploitability, weaknesses (CWE)
  │                 deduplicate by cve_id
  │                 time.sleep(0.6)   # respect NVD rate limit
  │         save all_cves → cve_details.csv
  │         Returns: List[dict]  (1,000 - 36,000 CVEs)
  │
  ├── embeddings = HuggingFaceEmbeddings(EMBED_MODEL)
  │
  ├── preprocess_and_build_knowledge_base(
  │      cves, CVE_JSONL, embeddings, INDEX_NAME)
  │         ├── write each CVE as JSON line → cve_knowledge_base.jsonl
  │         ├── pinecone.init() + create_index if needed
  │         ├── convert CVEs to LangChain Documents:
  │         │     page_content = cve['description']   # searchable field
  │         │     metadata     = {cve_id, severity, exploitability, weaknesses}
  │         └── LangchainPinecone.from_documents(docs, embeddings, index_name)
  │               └── Returns: VectorStore
  │
  ├── load_llm()      # same as Task 1
  ├── build_rag_chain(vector_store, llm)
```

```
    │
    └── for question in TASK2_QUESTIONS:
          ├── query_base_llm(llm, question)   # baseline
          └── query_rag_llm(rag_chain, question)
                ├── embed question → 384-dim vector
                ├── Pinecone search → top-5 CVE descriptions
                ├── Llama 2 synthesises threat assessment
                └── Returns: {answer, cve_references: [CVE-XXXX-YYYY, ...]}
```

C2. extract_keywords_from_pdf()

Signature	`extract_keywords_from_pdf(pdf_path: str) → List[str]`
Purpose	Extracts the top 15 domain-specific keyphrases from a single PDF design document using KeyBERT. These keyphrases are later used as NVD search terms. Two stopword lists are applied before extraction to ensure only meaningful, specific terms are returned.
Returns	List[str] — up to 15 keyphrases e.g. ['load value injection', 'trust domain extensions', 'memory encryption engine'].
Why it exists	Generic security terms would return all CVEs — useless for targeted threat modeling. Expert stopwords force KeyBERT to return technology-specific terms that give precise NVD results.

Internal control flow

```
1. PdfReader(pdf_path)
2. full_text = join(page.extract_text() for each page)
3. nltk_sw  = set(stopwords.words('english'))
4. all_sw   = list(nltk_sw | EXPERT_STOPWORDS)
     EXPERT_STOPWORDS = {security, attack, hacker, cyberattack,
       cyber, cybersecurity, intelligence, architecture, secure,
       encrypt, vulnerability, threat, network, system, ...}
     Why? Searching NVD for 'security' returns ALL 231K CVEs — useless.
     Searching for 'load value injection' returns specific, relevant CVEs.
5. kw_model = KeyBERT()
6. raw_keywords = kw_model.extract_keywords(
     full_text, keyphrase_ngram_range=(1,3),
     stop_words=all_sw, top_n=15,
     use_mmr=True, diversity=0.5)
7. PorterStemmer deduplication loop:
     for kw, score in raw_keywords:
         stem = stemmer.stem(kw.split()[0])
         if stem not in seen_stems: add to keywords list
Returns: List[str]  (up to 15 unique keyphrases)
```

Parameters

Parameter	Description
pdf_path	Absolute or relative path to a single PDF file.

Code

```
EXPERT_STOPWORDS = {
    'allow', 'attack', 'hacker', 'security', 'cyberattack', 'cyber',
    'cybersecurity', 'intelligence', 'architecture', 'secure', 'threat',
    'vulnerability', 'exploit', 'network', 'system', 'protocol', ...
}


def extract_keywords_from_pdf(pdf_path: str):
    reader    = PdfReader(pdf_path)
    full_text = ' '.join(page.extract_text() or '' for page in reader.pages)
    nltk_sw   = set(stopwords.words('english'))
    all_sw    = list(nltk_sw | EXPERT_STOPWORDS)
    kw_model  = KeyBERT()
    raw_kws   = kw_model.extract_keywords(
        full_text, keyphrase_ngram_range=(1, 3),
        stop_words=all_sw, top_n=15, use_mmr=True, diversity=0.5)
    stemmer = PorterStemmer()
    seen, keywords = set(), []
    for kw, score in raw_kws:
        stem = stemmer.stem(kw.split()[0])
        if stem not in seen:
            seen.add(stem)
            keywords.append(kw)
    return keywords
```

C3. extract_keywords_from_all_docs()

Signature	`extract_keywords_from_all_docs(docs_dir: str, output_csv: str) → List[str]`
Purpose	Iterates over all PDF files in the documents directory, calls extract_keywords_from_pdf() on each, aggregates results, deduplicates across documents, saves to a CSV file, and returns the unique keyword list for NVD querying.

Returns	List[str] — all unique keywords across all documents for NVD querying.
Why it exists	Different design documents may surface different keywords. By processing all documents and deduplicating, the NVD query set is comprehensive without redundant API calls.

Internal control flow

```
1. glob.glob(docs_dir + '/*.pdf')  →  list of PDF paths
2. for each pdf_path:
       keywords = extract_keywords_from_pdf(pdf_path)
       append {document: filename, keyword: kw} to all_keywords
3. unique_kws = set of all keyword strings
4. write all_keywords to CSV (document, keyword columns)
5. return unique_kws as list
```

Parameters

Parameter	Description
docs_dir	Directory containing PDF files.
output_csv	Path to write keywords.csv — each row: document name, keyword.

Code

```
def extract_keywords_from_all_docs(docs_dir, output_csv):
    os.makedirs(os.path.dirname(output_csv), exist_ok=True)
    all_keywords = []
    for pdf_path in glob.glob(os.path.join(docs_dir, '*.pdf')):
        kws = extract_keywords_from_pdf(pdf_path)
        for kw in kws:
            all_keywords.append({'document': os.path.basename(pdf_path),
                                 'keyword': kw})
    unique_kws = list({e['keyword'] for e in all_keywords})
    with open(output_csv, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=['document','keyword'])
        writer.writeheader()
        writer.writerows(all_keywords)
    return unique_kws
```

C4. query_nvd_api()

Signature	`query_nvd_api(keyword: str, api_key: str = '') → List[dict]`
Purpose	Makes a single HTTP GET request to the NVD REST API v2.0 searching for CVEs whose descriptions contain the given keyword. Parses the JSON response and extracts the fields needed for threat modeling: ID, description, severity, exploitability, and CWE weaknesses.
Returns	List[dict] — each dict: {cve_id, description, severity, exploitability, weaknesses, keyword}
Why it exists	The NVD contains 231,000+ CVEs. Querying with specific keywords returns a focused, product-relevant subset (100–3,600 CVEs per keyword). Scripts always query the latest NVD data — knowledge base stays current automatically.

Internal control flow

```
1. Build request:
     url     = 'https://services.nvd.nist.gov/rest/json/cves/2.0'
     params  = {keywordSearch: keyword, resultsPerPage: 100}
     headers = {apiKey: api_key}  (if provided)
2. requests.get(url, params, headers, timeout=30)
3. resp.raise_for_status()   # error on 4xx/5xx
4. data = resp.json()
5. for item in data['vulnerabilities']:
       cve_id      = item['cve']['id']
       description = first English description
       severity    = CVSS v3.1 baseScore (fallback to v2)
       exploitability = cvssData.exploitabilityScore
       weaknesses  = list of CWE IDs
6. return list of CVE dicts
On exception: log error, return []
```

Parameters

Parameter	Description
keyword	A single keyphrase from KeyBERT extraction e.g. 'load value injection'.
api_key	Optional NVD API key. Without it: 5 requests/30s limit. With it: 50 requests/30s.

Code

```
def query_nvd_api(keyword: str, api_key: str = '') -> list:
    url     = 'https://services.nvd.nist.gov/rest/json/cves/2.0'
    headers = {'apiKey': api_key} if api_key else {}
    params  = {'keywordSearch': keyword, 'resultsPerPage': 100}
```

```
    cves    = []
    try:
        resp = requests.get(url, params=params, headers=headers, timeout=30)
        resp.raise_for_status()
        for item in resp.json().get('vulnerabilities', []):
            cve      = item.get('cve', {})
            descs    = cve.get('descriptions', [])
            desc     = next((d['value'] for d in descs
                             if d.get('lang')=='en'), '')
            metrics  = cve.get('metrics', {})
            cvss     = (metrics.get('cvssMetricV31',[{}])[0]
                        .get('cvssData', {}))
            cves.append({
                'cve_id':         cve.get('id',''),
                'description':    desc,
                'severity':       cvss.get('baseScore','N/A'),
                'exploitability': cvss.get('exploitabilityScore','N/A'),
                'weaknesses':     '; '.join([
                    p['description'][0]['value']
                    for p in cve.get('weaknesses',[]) if p.get('description')]),
                'keyword':        keyword,
            })
    except requests.exceptions.RequestException as e:
        print(f'NVD error for {keyword}: {e}')
    return cves
```

C5. build_cve_dataset()

Signature	`build_cve_dataset(keywords, output_csv, api_key='') → List[dict]`
Purpose	Iterates over all extracted keywords, calls query_nvd_api() for each, deduplicates results by CVE ID (same CVE may match multiple keywords), saves to CSV, and returns the full custom vulnerability dataset. This dataset is the raw material for the Task 2 knowledge base.

Returns	List[dict] — deduplicated CVE records, 1,000–36,000 entries depending on design document scope.
Why it exists	Deduplication is critical — 'encryption' and 'encrypted channel' might both return CVE-2023-1234. Without deduplication the vector database would have duplicate entries inflating results.

Internal control flow

```
1. all_cves = [], seen_ids = set()
2. for keyword in keywords:
       cves = query_nvd_api(keyword, api_key)
       for cve in cves:
           if cve['cve_id'] not in seen_ids:
               seen_ids.add(cve['cve_id'])
               all_cves.append(cve)
       time.sleep(0.6)   # NVD rate limit
3. write all_cves to CSV
4. return all_cves
Result size: 1,000 (narrow docs) to 36,000 (broad docs) unique CVEs
```

Parameters

Parameter	Description
keywords	List[str] — output of extract_keywords_from_all_docs().
output_csv	Path to write cve_details.csv.
api_key	Optional NVD API key for higher rate limits.

Code

```
def build_cve_dataset(keywords, output_csv, api_key=''):

    os.makedirs(os.path.dirname(output_csv), exist_ok=True)

    all_cves, seen_ids = [], set()
```

```
    for i, keyword in enumerate(keywords, 1):

        cves = query_nvd_api(keyword, api_key)

        for cve in cves:

            if cve['cve_id'] not in seen_ids:

                seen_ids.add(cve['cve_id'])
```

```
                all_cves.append(cve)
```

```
        time.sleep(0.6)          # respect NVD rate limit

    with open(output_csv, 'w', newline='') as f:

        writer = csv.DictWriter(f, fieldnames=all_cves[0].keys())
```

```
        writer.writeheader(); writer.writerows(all_cves)
    return all_cves
```

C6. preprocess_and_build_knowledge_base()

Signature	`preprocess_and_build_knowledge_base(cves, jsonl_path, embeddings, index_name) → VectorStore`
Purpose	Converts the raw CVE dataset into a Pinecone vector knowledge base. Three sub-steps: (1) write CVEs as JSONL file, (2) convert each CVE to a LangChain Document with the description as searchable content, (3) embed all descriptions and index them in Pinecone.
Returns	LangchainPinecone VectorStore — CVE knowledge base ready for semantic search.
Why it exists	CVE descriptions are embedded (not keyword-matched) so semantically related CVEs are retrieved even when the exact keyword is absent. A query about 'key exposure' retrieves CVEs describing 'private key disclosure' without requiring the exact phrase.

Internal control flow

```
1. Write JSONL file (Steps 2-3):
     with open(jsonl_path, 'w') as f:
         for cve in cves: f.write(json.dumps(cve) + '\n')
     JSONL advantages: stream processing, error isolation,
     space efficiency (no commas between objects)
2. pinecone.init() + create_index if needed
3. Convert CVEs to Documents (Step 2-4):
     for cve in cves:
         Document(
           page_content = cve['description'],   # ← searchable by BERT
           metadata = {cve_id, severity,
                       exploitability, weaknesses, keyword}
         )
4. LangchainPinecone.from_documents(documents, embeddings, index_name)
     ↓  embed each description → 384-dim vector
     ↓  upsert (vector, metadata) into Pinecone
Returns: VectorStore
```

Parameters

Parameter	Description
cves	List[dict] — output of build_cve_dataset().
jsonl_path	Path to write cve_knowledge_base.jsonl — intermediate storage.
embeddings	HuggingFaceEmbeddings — for computing CVE description vectors.
index_name	Pinecone index name for Task 2 CVE knowledge base.

Code

```
def preprocess_and_build_knowledge_base(cves, jsonl_path, embeddings, index_name):
    os.makedirs(os.path.dirname(jsonl_path), exist_ok=True)
    # Step 2-3: JSONL
    with open(jsonl_path, 'w') as f:
        for cve in cves:
            f.write(json.dumps(cve) + '\n')
    # Pinecone setup
    pinecone.init(api_key=PINECONE_API_KEY, environment=PINECONE_ENV)
    if index_name not in pinecone.list_indexes():
        pinecone.create_index(index_name, dimension=384, metric='cosine')
    # Step 2-4: Build vector DB
    documents = [
        Document(
            page_content=cve['description'],
            metadata={
                'cve_id':         cve['cve_id'],
                'severity':       str(cve['severity']),
                'exploitability': str(cve['exploitability']),
                'weaknesses':     cve['weaknesses'],
                'keyword':        cve['keyword'],
            },
        )
        for cve in cves if cve['description']
    ]
    return LangchainPinecone.from_documents(documents, embeddings,
index_name=index_name)
```

D Cross-Cutting Concerns

RAG internals · Pinecone mechanics · Configuration reference · End-to-end data flow

D1. RAG vs Base LLM — Mechanics

Base LLM (no retrieval)

Prompt sent:

```
'You are a security engineer.'

'Answer: {question}'
```

- Llama 2 answers from training data only
- No product-specific context available
- May hallucinate product details
- Average: ~253 words per response
- Confident even when wrong

RAG-Enhanced LLM

Prompt sent:

```
'Use ONLY this context to answer.'

'Context: [chunk1][chunk2][chunk3]'

'[chunk4][chunk5]'

'If context insufficient: say I dont know'

'Question: {question}'
```

- Llama 2 answers from retrieved chunks only
- Product-specific, grounded in real docs
- Hallucinations greatly reduced
- Average: ~74 words per response
- Explicitly says 'I don't know' when context insufficient

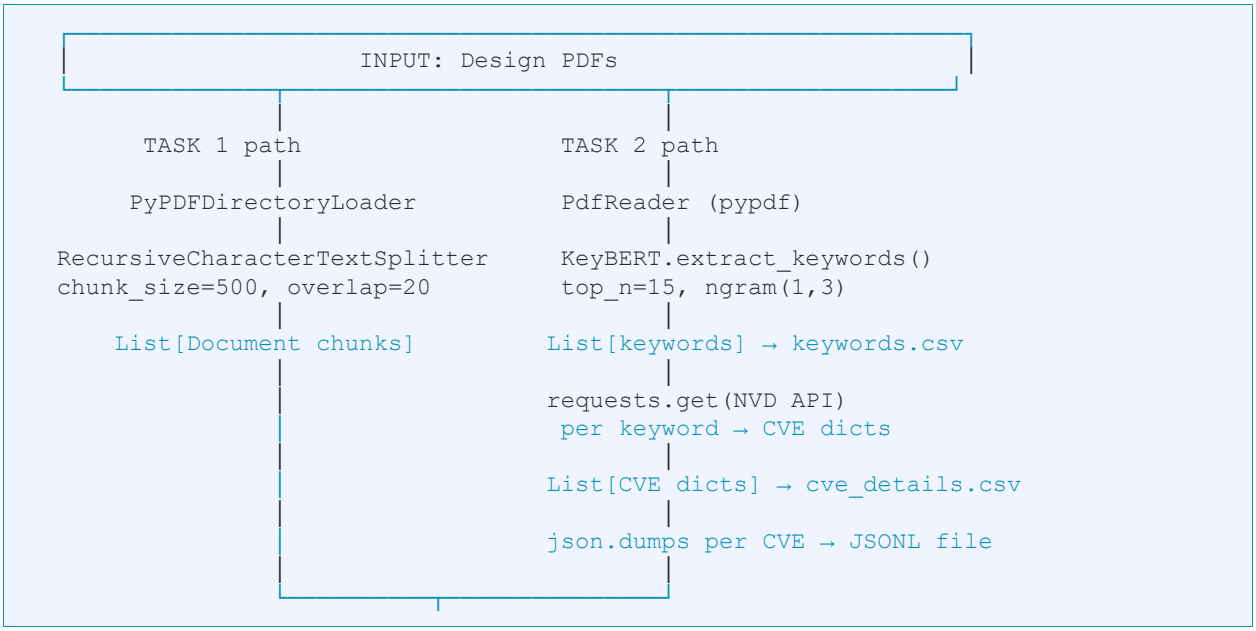
D2. Pinecone Index Internals

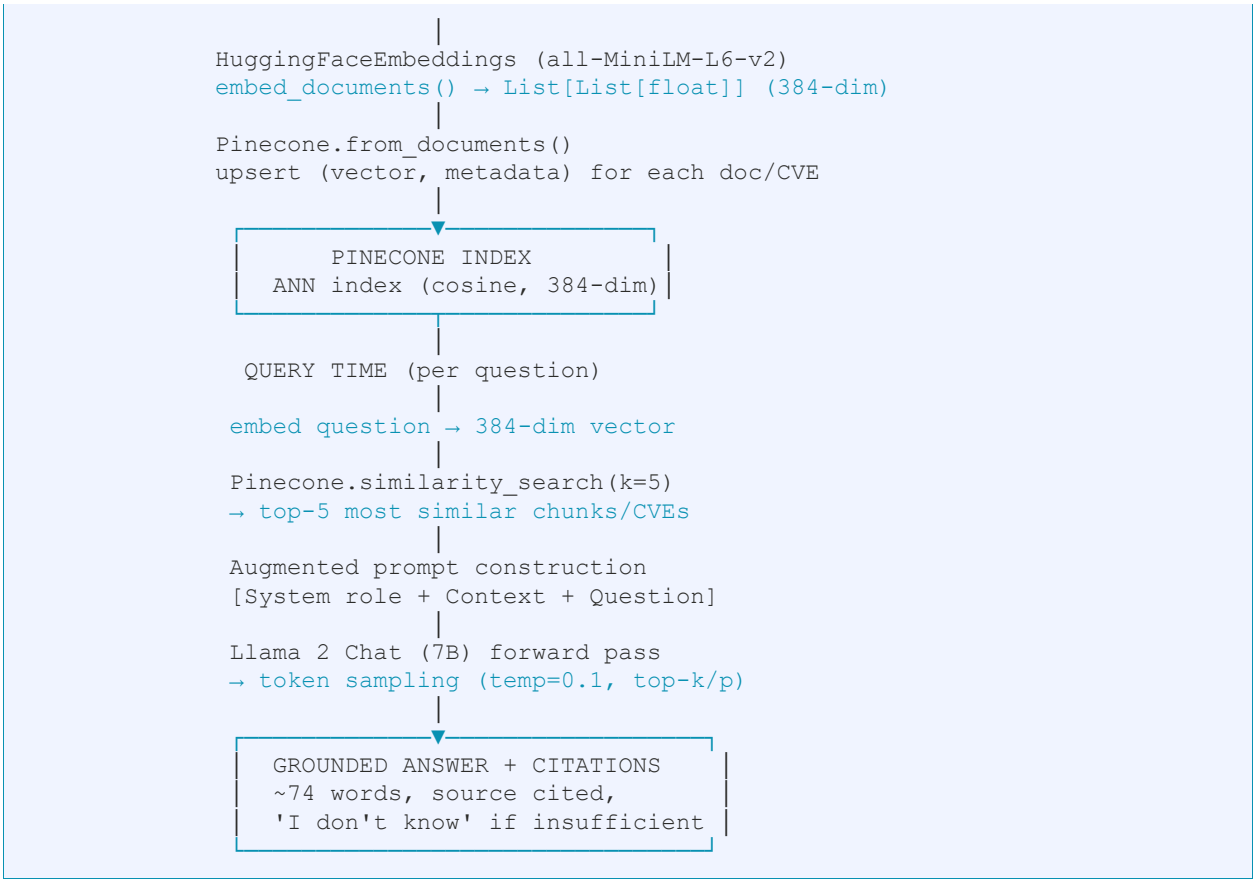
Property	Task 1 (Documents)	Task 2 (CVEs)
Index name	threat-modeling-task1	threat-modeling-task2
Embedding model	all-MiniLM-L6-v2	all-MiniLM-L6-v2 (same)
Vector dimension	384	384
Similarity metric	cosine	cosine
Stored content	Design doc text chunks	CVE descriptions
Metadata per vector	source file, page number	cve_id, severity, exploitability, CWE, keyword
Typical index size	~4,200 vectors (12 docs)	1,000 – 36,000 vectors
Query top-k	5 chunks	5 CVEs

D3. Configuration Constants Reference

Constant	Value	Why this value
CHUNK_SIZE	500 chars	~3-4 sentences — small enough to be specific, large enough for context.
CHUNK_OVERLAP	20 chars	Prevents losing a sentence split at a chunk boundary.
TOP_K	5	Retrieves 5 chunks × 500 chars = 2,500 chars — fits in Llama 2 context window.
KEYBERT_TOP_N	15	15 keyphrases per document provides enough NVD coverage without too many generic terms.
NGRAM_RANGE	(1,3)	Allows 1, 2, and 3-word keyphrases. 3-word phrases are far more specific for NVD.
MMR_DIVERSITY	0.5	Balances relevance and variety — avoids 5 synonyms of 'encryption'.
NVD_RESULTS_PP	100	Maximum per API call. Rate limits require batching.
NVD_SLEEP	0.6s	Stays below NVD's 5 requests/30s limit without API key.
TEMPERATURE	0.1	Low temperature = deterministic responses suitable for factual Q&A.
MAX_NEW_TOKENS	512	Caps response length. 512 tokens ≈ 384 words.
LLM	Llama-2-7b-chat-hf	7B parameter chat-tuned model. Paper also tests 13B.

D4. End-to-End Data Flow — Both Tasks





Summary — why this architecture works:

Every design decision compounds: KeyBERT extracts specific (not generic) keywords → NVD returns relevant (not all) CVEs → BERT embeddings capture semantic (not keyword) similarity → Pinecone retrieves contextually closest chunks → Llama 2 generates grounded (not hallucinated) answers. Remove any one component and quality degrades measurably.